



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



LAB MANUAL: FILE AND FOLDER AUTOMATION USING PYTHON SHUTIL MODULE

Lab Title

File and Folder Automation Using the Python `shutil` Module

Lab Objectives

The objective of this lab is to understand how Python can automate common file and folder management operations using the `shutil` module. Students will learn how to copy, move, organize, delete, archive, and extract files and directories programmatically.

By the end of this lab, students will be able to:

- Understand the purpose of the `shutil` module.
 - Copy files from one location to another.
 - Copy files while preserving metadata.
 - Copy complete directories and their contents.
 - Move and rename files and folders.
 - Delete complete directory trees.
 - Check disk usage using Python.
 - Find executable programs available on the system.
 - Create compressed archives.
 - Extract compressed archives.
 - Handle errors during file operations.
 - Combine `shutil` with `os` and `pathlib`.
 - Build a simple automated file organization system.
-



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



1. Introduction to File Automation

File management is one of the most common tasks performed on a computer. Users regularly create folders, copy files, move documents, organize downloads, delete temporary data, and create backups.

When these operations involve hundreds or thousands of files, performing them manually becomes time-consuming and repetitive.

Python allows us to automate these operations.

For example, a Python program can automatically:

- Move PDF files into a Documents folder.
- Move images into an Images folder.
- Copy important files into a Backup folder.
- Create compressed ZIP backups.
- Delete temporary directories.
- Extract downloaded archives.
- Check available disk space.
- Organize files according to their extensions.

Python provides several modules for working with files and directories. One of the most useful modules for high-level file operations is `shutil`.

The name `shutil` comes from **Shell Utilities**.

The module is included in the Python Standard Library, which means that no additional installation is required.

We can import it using:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
iimport shutil
```

2. Understanding os, pathlib, and shutil

Python provides multiple modules for working with files and directories. Three commonly used modules are **os**, **pathlib**, and **shutil**.

The **os** module provides operating-system-related functionality. It can be used to create directories, remove individual files, access environment variables, and perform many low-level system operations.

The **pathlib** module provides a modern, object-oriented way to work with file and directory paths. It makes path manipulation easier and improves code readability.

The **shutil** module provides high-level file operations. It is especially useful when we need to copy, move, delete, or archive files and complete directories.

In real automation projects, these modules are often used together.

Example:

Python

```
iimport os
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
import shutil  
  
from pathlib import Path
```

Here, **Path** can manage paths, while **shutil** performs the actual copying or moving operation.

3. Setting Up the Lab Environment

Before performing the practical exercises, create a folder for this lab.

The suggested folder structure is:

```
shutil_lab/  
|  
|— source/  
| |— document.txt  
| |— report.pdf  
| |— image.jpg  
|  
|— destination/  
|
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



└─ main.py

Create the folders manually or use Python.

Example:

Python

```
from pathlib import Path
```

```
Path("source").mkdir(exist_ok=True)
```

```
Path("destination").mkdir(exist_ok=True)
```

```
print("Lab directories created successfully.")
```

The `exist_ok=True` argument prevents Python from generating an error if the directory already exists.

4. Practical 1: Copying a File Using `shutil.copy()`

The `shutil.copy()` function is used to copy a file from one location to another.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Basic syntax:

```
shutil.copy(source, destination)
```

The **source** represents the original file, while **destination** represents the location where the file should be copied.

Example:

Python

```
import shutil
```

```
source = "source/document.txt"
```

```
destination = "destination/document.txt"
```

```
shutil.copy(source, destination)
```

```
print("File copied successfully.")
```

After running the program, **document.txt** will remain inside the **source** folder, and another copy will be created inside the **destination** folder.

The original file is not removed.



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Task

Create a text file named `notes.txt` inside the `source` directory and copy it into the `destination` directory using `shutil.copy()`.

5. Practical 2: Copying and Renaming a File

The `copy()` function can also copy a file while giving the copied file a different name.

Example:

Python

```
import shutil
```

```
source = "source/document.txt"
```

```
destination = "destination/document_backup.txt"
```

```
shutil.copy(source, destination)
```

```
print("File copied and renamed successfully.")
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



The original file remains:

source/document.txt

The copied file becomes:

destination/document_backup.txt

This technique is useful when creating simple backup copies.

Task

Copy `notes.txt` from the `source` folder to the `destination` folder and rename the copied file to `notes_backup.txt`.

6. Practical 3: Copying Files with Metadata Using `shutil.copy2()`

The `shutil.copy2()` function works similarly to `shutil.copy()`, but it attempts to preserve additional file metadata.

Metadata may include information such as:

- File modification time.
- File access time.
- File permission information.

Example:



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



Python

```
import shutil
```

```
source = "source/document.txt"
```

```
destination = "destination/document.txt"
```

```
shutil.copy2(source, destination)
```

```
print("File copied with metadata.")
```

For normal file copying, `copy()` may be sufficient. For backup systems where preserving file information is important, `copy2()` is generally more suitable.

Task

Use `shutil.copy2()` to create a backup copy of `report.pdf`.

7. Practical 4: Copying File Permissions Using `shutil.copymode()`

The `shutil.copymode()` function copies permission bits from one file to another.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



It does not copy the contents of the file.

Syntax:

```
shutil.copymode(source, destination)
```

Example:

Python

```
import shutil
```

```
shutil.copymode(  
    "source/document.txt",  
    "destination/document.txt"  
)
```

```
print("File permissions copied.")
```

This operation is particularly useful on operating systems such as Linux, where file permissions are an important part of system administration.



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



8. Practical 5: Copying File Metadata Using `shutil.copystat()`

The `shutil.copystat()` function copies file status information from one file to another.

This may include:

- Permission bits.
- Last access time.
- Last modification time.
- Other supported metadata.

Example:

Python

```
import shutil

shutil.copystat(
    "source/document.txt",
    "destination/document.txt"
)

print("File metadata copied.")
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



The difference is that `copymode()` mainly focuses on permission information, while `copystat()` copies additional file status information.

9. Practical 6: Copying Complete Directories Using `shutil.copytree()`

Sometimes we need to copy an entire directory rather than a single file.

The `shutil.copytree()` function copies a directory and all of its contents.

Example:

```
Python
import shutil

source = "source"

destination = "source_backup"

shutil.copytree(source, destination)

print("Directory copied successfully.")
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



If the destination directory already exists, the operation may generate an error.

Modern Python versions allow:

```
shutil.copytree(  
    "source",  
    "source_backup",  
    dirs_exist_ok=True  
)
```

This allows files to be copied into an existing destination directory.

Task

Create a folder containing at least three files and copy the complete folder using `shutil.copytree()`.

10. Practical 7: Ignoring Files While Copying Directories

Sometimes we do not want to copy every file.

For example, a project directory may contain temporary files or Python cache directories that should not be included in a backup.

The `ignore_patterns()` function can be used with `copytree()`.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Example:

Python

```
import shutil

shutil.copytree(

    "source",

    "backup",

    ignore=shutil.ignore_patterns(

        "*.tmp",

        "*.log",

        "__pycache__"

    ),

    dirs_exist_ok=True

)

print("Selective backup completed.")
```

In this example:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



- `.tmp` files are ignored.
- `.log` files are ignored.
- `__pycache__` directories are ignored.

This is useful when creating clean project backups.

11. Practical 8: Moving Files Using `shutil.move()`

The `shutil.move()` function moves a file from one location to another.

Unlike copying, moving normally removes the file from its original location.

Example:

Python

```
import shutil
```

```
source = "source/document.txt"
```

```
destination = "destination/document.txt"
```

```
shutil.move(source, destination)
```

```
print("File moved successfully.")
```



Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro



After the operation, the file will no longer exist in the original `source` directory.

Task

Create a file named `example.txt` and move it from the `source` folder into the `destination` folder.

12. Practical 9: Renaming Files Using `shutil.move()`

The `move()` function can also be used to rename a file.

Example:

Python

```
import shutil
```

```
shutil.move(
```

```
    "old_name.txt",
```

```
    "new_name.txt"
```




**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
)
```

```
print("File renamed successfully.")
```

This works because moving a file to a new path within the same directory effectively changes its name.

13. Practical 10: Moving Complete Directories

The `move()` function is not limited to individual files. It can also move complete directories.

Example:

```
Python
```

```
import shutil
```

```
shutil.move(
```

```
    "project_folder",
```

```
    "backup/project_folder"
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
)
```

```
print("Directory moved successfully.")
```

All files and subdirectories inside `project_folder` are moved along with the directory.

This is useful in automated project management and backup workflows.

14. Practical 11: Deleting Directories Using `shutil.rmtree()`

The `shutil.rmtree()` function removes a complete directory tree.

Example:

```
Python
```

```
import shutil
```

```
shutil.rmtree("temporary_folder")
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
print("Directory deleted successfully.")
```

This operation deletes:

- The target directory.
- All files inside it.
- All subdirectories.
- All files inside the subdirectories.

Therefore, `rmtree()` should be used carefully.

A safer approach is to check whether the directory exists first.

Python

```
import shutil
```

```
from pathlib import Path
```

```
folder = Path("temporary_folder")
```

```
if folder.exists() and folder.is_dir():
```

```
    shutil.rmtree(folder)
```

```
    print("Directory deleted.")
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



else:

```
print("Directory does not exist.")
```

Never use `rmtree()` on an unknown path without validating it first.

15. Practical 12: Checking Disk Usage

The `shutil.disk_usage()` function provides information about storage usage.

Example:

Python

```
import shutil
```

```
usage = shutil.disk_usage("/")
```

```
print("Total:", usage.total)
```

```
print("Used:", usage.used)
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
print("Free:", usage.free)
```

The returned values are measured in bytes.

We can convert them into gigabytes.

Python

```
import shutil
```

```
usage = shutil.disk_usage("/")
```

```
gb = 1024 ** 3
```

```
print("Total:", round(usage.total / gb, 2), "GB")
```

```
print("Used:", round(usage.used / gb, 2), "GB")
```

```
print("Free:", round(usage.free / gb, 2), "GB")
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



This feature can be used in automation systems to check whether enough storage space is available before creating large backups.

16. Practical 13: Finding Installed Commands Using `shutil.which()`

The `shutil.which()` function checks whether an executable command is available in the system's `PATH`.

Example:

```
Python
import shutil

python_path = shutil.which("python")

print(python_path)
```

We can check multiple programs:



Python

```
import shutil
```

```
programs = ["python", "git", "code"]
```

```
for program in programs:
```

```
    path = shutil.which(program)
```

```
    if path:
```

```
        print(program, "found at:", path)
```

```
    else:
```

```
        print(program, "is not installed or not in PATH")
```

This is useful when an automation script depends on external software.

17. Practical 14: Creating ZIP Archives

The `shutil.make_archive()` function can create compressed archives.

Example:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Python

```
import shutil
```

```
shutil.make_archive(
```

```
    "project_backup",
```

```
    "zip",
```

```
    "source"
```

```
)
```

```
print("ZIP archive created successfully.")
```

This creates:

project_backup.zip

The parameters represent:

Archive Name → project_backup

Archive Format → zip

Source Directory → source



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Compressed archives are commonly used for backups and transferring multiple files.

18. Practical 15: Extracting Archives

The `shutil.unpack_archive()` function extracts compressed archives.

Example:

Python

```
import shutil

shutil.unpack_archive(
    "project_backup.zip",
    "extracted_files"
)

print("Archive extracted successfully.")
```

The files inside the archive will be extracted into:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



extracted_files/

This can be useful when automatically processing downloaded datasets, software packages, or project files.

19. Practical 16: Viewing Supported Archive Formats

Python can display the archive formats supported by the current environment.

Example:

```
Python
import shutil

formats = shutil.get_archive_formats()

for name, description in formats:
    print(name, "-", description)
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Depending on the Python installation and operating system, common formats may include ZIP and different TAR-based formats.

20. Error Handling in File Automation

File operations can fail for several reasons.

Common problems include:

- The source file does not exist.
- The destination path is incorrect.
- The user does not have permission.
- The destination file already exists.
- The disk does not have enough free space.
- Another program is using the file.

We should use exception handling when building reliable automation programs.

Example:

Python

```
import shutil
```

```
try:
```

```
    shutil.copy(
```

```
        "source/document.txt",
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
"destination/document.txt"
)

print("File copied successfully.")

except FileNotFoundError:

    print("Error: Source file was not found.")

except PermissionError:

    print("Error: Permission denied.")

except OSError as error:

    print("System error:", error)
```

Using exception handling prevents the complete program from crashing unexpectedly.



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



21. Combining pathlib with shutil

The `pathlib` module provides a cleaner way to work with paths.

Example:

```
Python
from pathlib import Path

import shutil

source = Path("source")
destination = Path("destination")

destination.mkdir(exist_ok=True)

file = source / "document.txt"

shutil.copy(
    file,
    destination / file.name
)
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
print("File copied successfully.")
```

Using `pathlib` is recommended for modern Python applications because it makes path-related code easier to read and maintain.

22. Practical Automation Project: Automatic File Organizer

In this project, we will create a simple automation program that organizes files according to their extensions.

Suppose we have the following files:

Downloads/

|

|— photo.jpg

|— logo.png

|— report.pdf

|— notes.txt

|— video.mp4

|— data.csv



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



└─ project.zip

We want the program to automatically create the following structure:

Downloads/

|

└─ Images/

| └─ photo.jpg

| └─ logo.png

|

└─ Documents/

| └─ report.pdf

| └─ notes.txt

|

└─ Videos/

| └─ video.mp4

|

└─ Data/

| └─ data.csv

|

└─ Archives/

└─ project.zip



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



The automation program is:

```
Python
from pathlib import Path

import shutil

source_folder = Path("Downloads")

categories = {

    "Images": [".jpg", ".jpeg", ".png", ".gif"],

    "Documents": [".pdf", ".txt", ".docx"],

    "Videos": [".mp4", ".mkv", ".avi"],

    "Data": [".csv", ".xlsx", ".json"],

    "Archives": [".zip", ".rar", ".tar"]

}

for file in source_folder.iterdir():

    if file.is_file():

        for folder_name, extensions in categories.items():

            if file.suffix.lower() in extensions:
```




```
destination = source_folder / folder_name

destination.mkdir(exist_ok=True)

shutil.move(

    str(file),

    str(destination / file.name)

)

print(f"Moved {file.name} to {folder_name}")

break
```

```
print("File organization completed.")
```

How the Program Works

Expected Output:

```
source_folder = Path("Downloads")
```

Next, a dictionary defines different categories and their associated file extensions.

For example:

```
"Images": [".jpg", ".jpeg", ".png", ".gif"]
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



The program then loops through every item in the directory.

```
for file in source_folder.iterdir():
```

It checks whether the item is a file.

```
if file.is_file():
```

The file extension is then compared with the extensions stored in the categories dictionary.

When a matching category is found, the program creates the required directory if it does not already exist.

```
destination.mkdir(exist_ok=True)
```

Finally, the file is moved into the correct directory using:

```
shutil.move()
```

This demonstrates how a repetitive manual file-management task can be converted into an automated workflow.



23. Practical Automation Project: Automated Backup System

The following program creates a timestamped backup of a folder.

Python

```
from pathlib import Path

from datetime import datetime

import shutil

source = Path("project")

backup_folder = Path("backups")

backup_folder.mkdir(exist_ok=True)

timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")

backup_name = backup_folder / f"project_{timestamp}"

shutil.copytree(source, backup_name)

print("Backup created successfully.")

print("Backup location:", backup_name)
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Each backup receives a unique name based on the current date and time.

Example:

backups/

|

|— project_2026-07-23_09-30-00/

|— project_2026-07-23_10-15-20/

|— project_2026-07-23_11-40-10/

This prevents previous backups from being overwritten.

24. Practical Automation Project: Compressed Backup System

We can improve the previous backup system by creating a compressed ZIP archive instead of copying the complete folder.

Python

```
from pathlib import Path
```

```
from datetime import datetime
```

```
import shutil
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
source = Path("project")

backup_folder = Path("backups")

backup_folder.mkdir(exist_ok=True)

timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
archive_name = backup_folder / f"project_{timestamp}"

shutil.make_archive(
    str(archive_name),
    "zip",
    str(source)
)

print("Compressed backup created successfully.")

print("Archive:", str(archive_name) + ".zip")
```

The resulting backups may look like:

backups/



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



```
|
|— project_2026-07-23_09-30-00.zip
|— project_2026-07-23_10-15-20.zip
|— project_2026-07-23_11-40-10.zip
```

This approach can reduce storage usage and make backups easier to transfer.

25. Lab Exercise

Create a Python-based **Smart File Management System**.

The program should:

1. Accept or define a folder that needs to be organized.
2. Scan all files inside the folder.
3. Detect each file's extension.
4. Create separate folders for Images, Documents, Videos, Data, and Archives.
5. Move files into their appropriate folders.
6. Create a backup before moving the files.
7. Compress the backup into a ZIP archive.
8. Display every file operation in the terminal.
9. Handle missing directories using exception handling.
10. Display a final message when the automation process is complete.

Expected Workflow

Start Program

```
|
```



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



v

Select Source Folder

|

v

Check Folder Exists

|

v

Create Backup

|

v

Compress Backup

|

v

Scan Files

|

v

Detect File Extensions

|

v

Create Category Folders

|



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



v

Move Files

|

v

Display Summary

|

v

End Program

Lab Assignment

Develop a **Download Folder Automation System** using Python.

Your program should automatically organize files into categories based on file extensions.

Required categories:

Images

Documents

Videos

Audio

Archives

Code



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Data

Others

Category	File Extensions
Images	.jpg, .jpeg, .png, .gif
Documents	.pdf, .txt, .docx
Videos	.mp4, .mkv, .avi
Data	.csv, .xlsx, .json
Archives	.zip, .rar, .tar

The program must use:

`shutil.move()`

`shutil.copytree()`

`shutil.make_archive()`

The program should also use `pathlib` for path management and `try-except` blocks for error handling.

Before organizing files, the system must create a backup of the original directory.

After the automation is complete, display a summary similar to:



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



Expected Summary Output:

None

===== AUTOMATION SUMMARY =====

Images moved: 5

Documents moved: 8

Videos moved: 2

Audio files moved: 3

Archives moved: 4

Code files moved: 6

Data files moved: 2

Other files moved: 1

Total files organized: 31

Backup created successfully.

File organization completed successfully.

Conclusion



**Benazir Bhutto Shaheed Human
Resource Research & Development
Board [BBSHRRDB]
PMU-ORIC University of Sindh,
Jamshoro**



In this lab, we learned how the Python `shutil` module can be used to automate file and directory management operations.

We used `shutil.copy()` and `shutil.copy2()` to copy files, `shutil.copytree()` to copy complete directories, and `shutil.move()` to move and organize files. We also learned how `shutil.rmtree()` can remove directory trees and how `shutil.disk_usage()` can retrieve information about available storage.

We explored Python's archive management features using `make_archive()` and `unpack_archive()`. Finally, we combined `shutil` with `pathlib`, `datetime`, and exception handling to create practical file organization and backup automation systems.

These concepts form the foundation for building more advanced automation applications such as automated backup tools, download organizers, file synchronization systems, data-processing pipelines, and scheduled maintenance scripts.